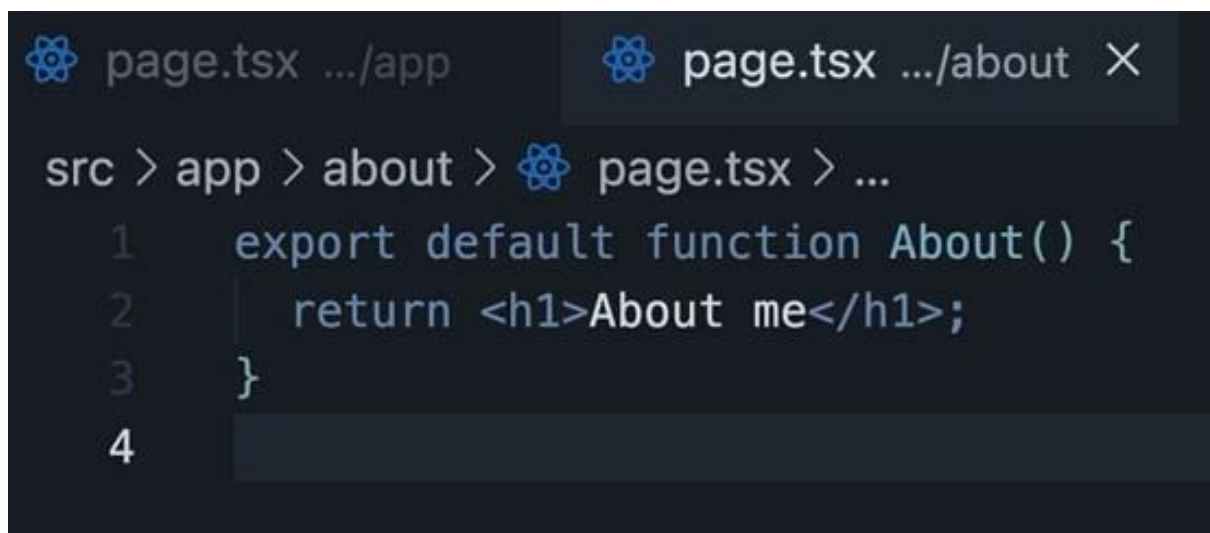
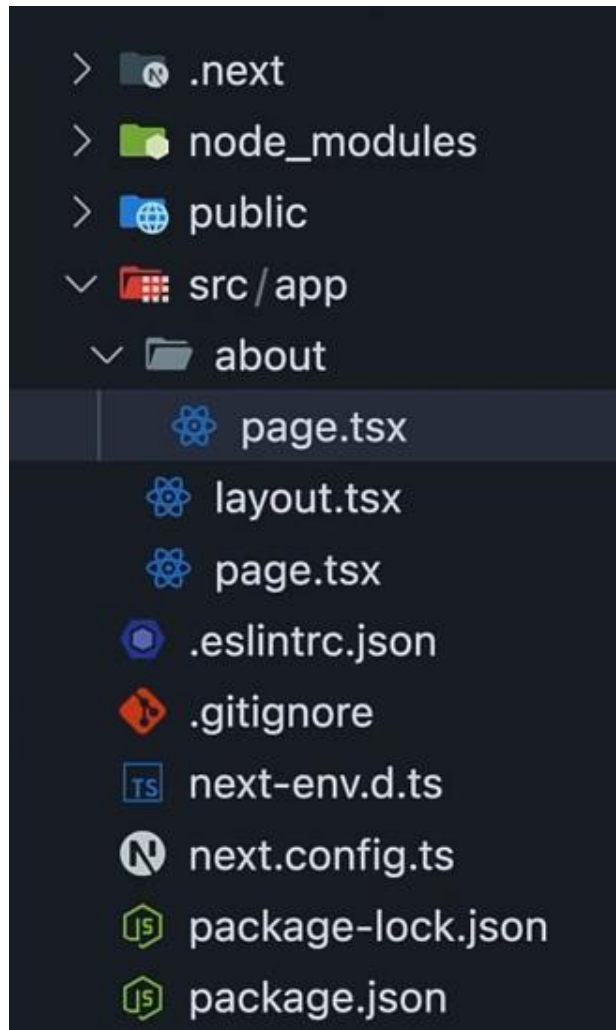
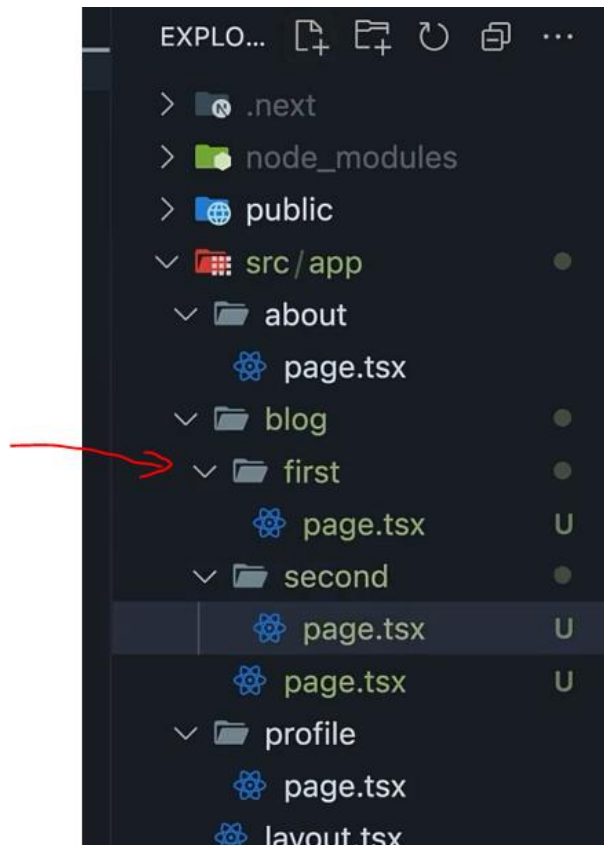


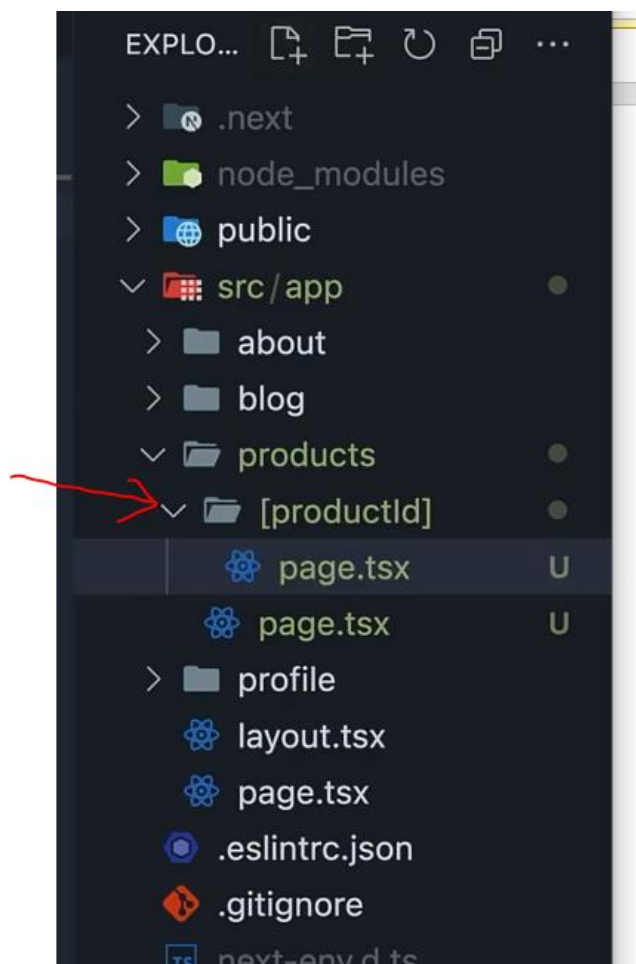
Nextjs routing



Nested Route

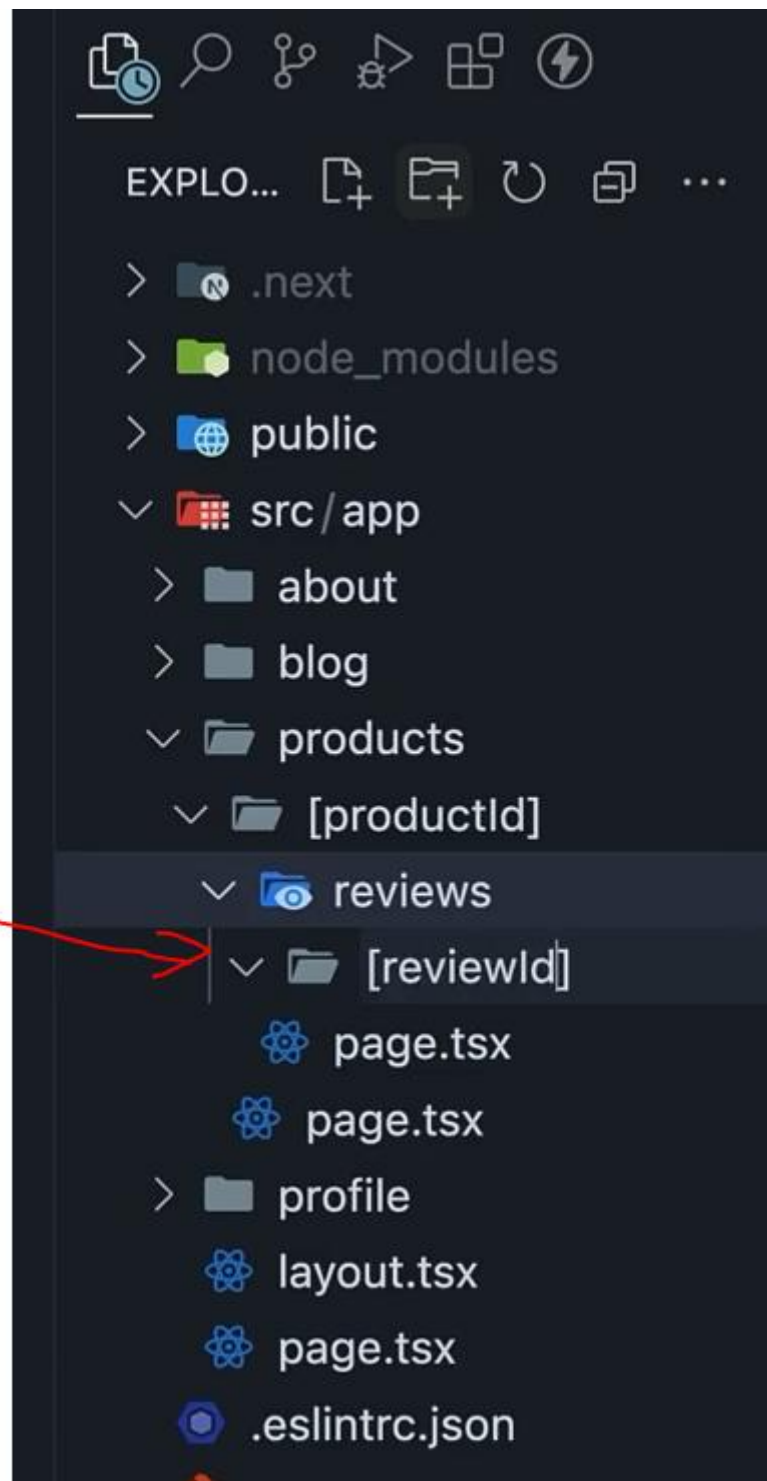


Dynamic Route



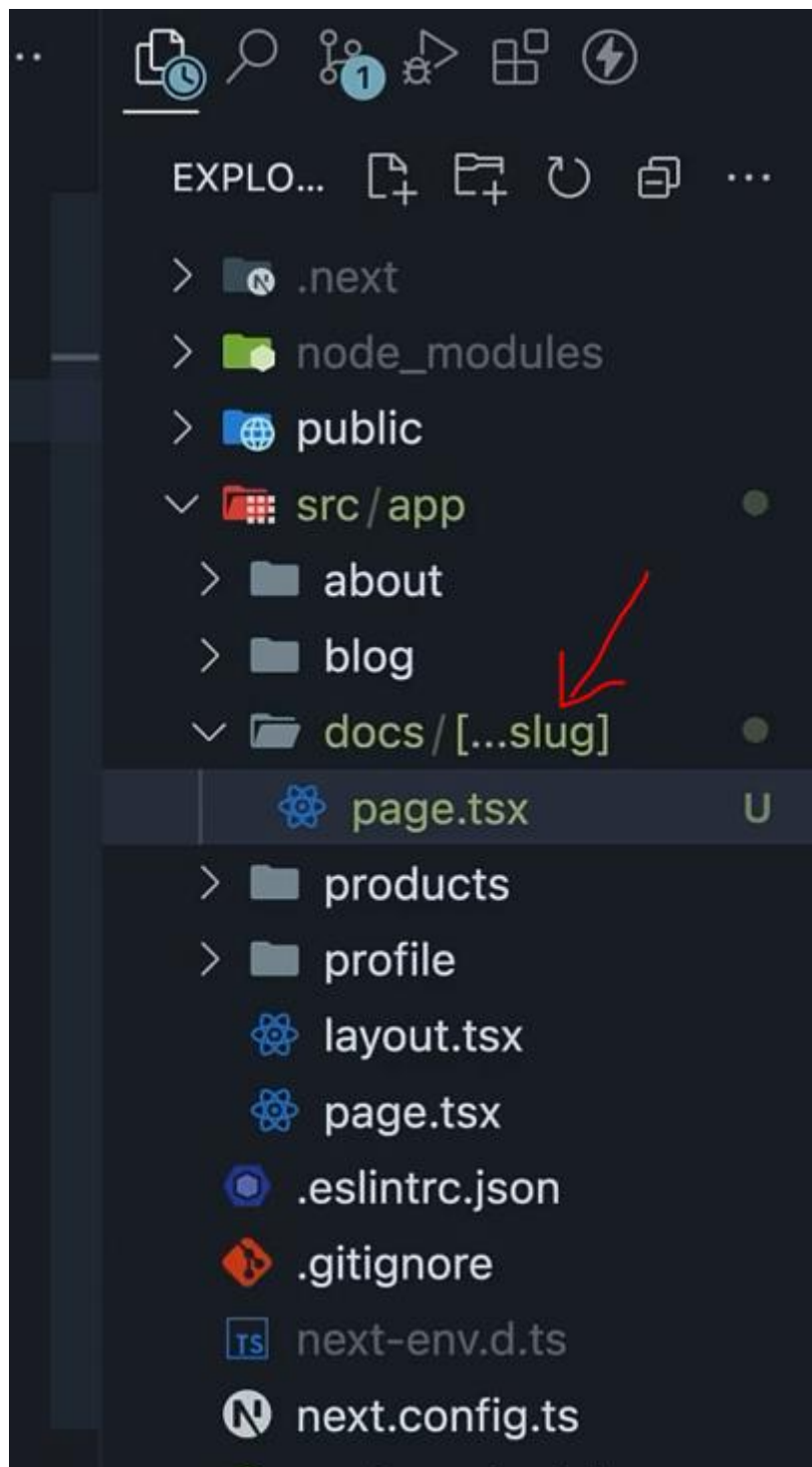
```
page.tsx .../products U  page.tsx .../[productId] U X
src > app > products > [productId] > page.tsx > ...
1  export default async function ProductDetails({
2    params,
3  }): {
4    params: Promise<{ productId: string }>;
5  } {
6    const productId = (await params).productId;
7    return <h1>Details about product {productId}</h1>;
8  }
9
```

Nested Dynamic Route



```
page.tsx U X
src > app > products > [productId] > reviews > [reviewId] > page.tsx > ...
1  export default async function ProductReview({
2    params,
3  }): {
4    params: Promise<{ productId: string; reviewId: string }>;
5  } {
6    const { productId, reviewId } = await params;
7    return (
8      <h1>
9        Review {reviewId} for product {productId}
10      </h1>
11    );
12  }
13
```

Catch All Segment

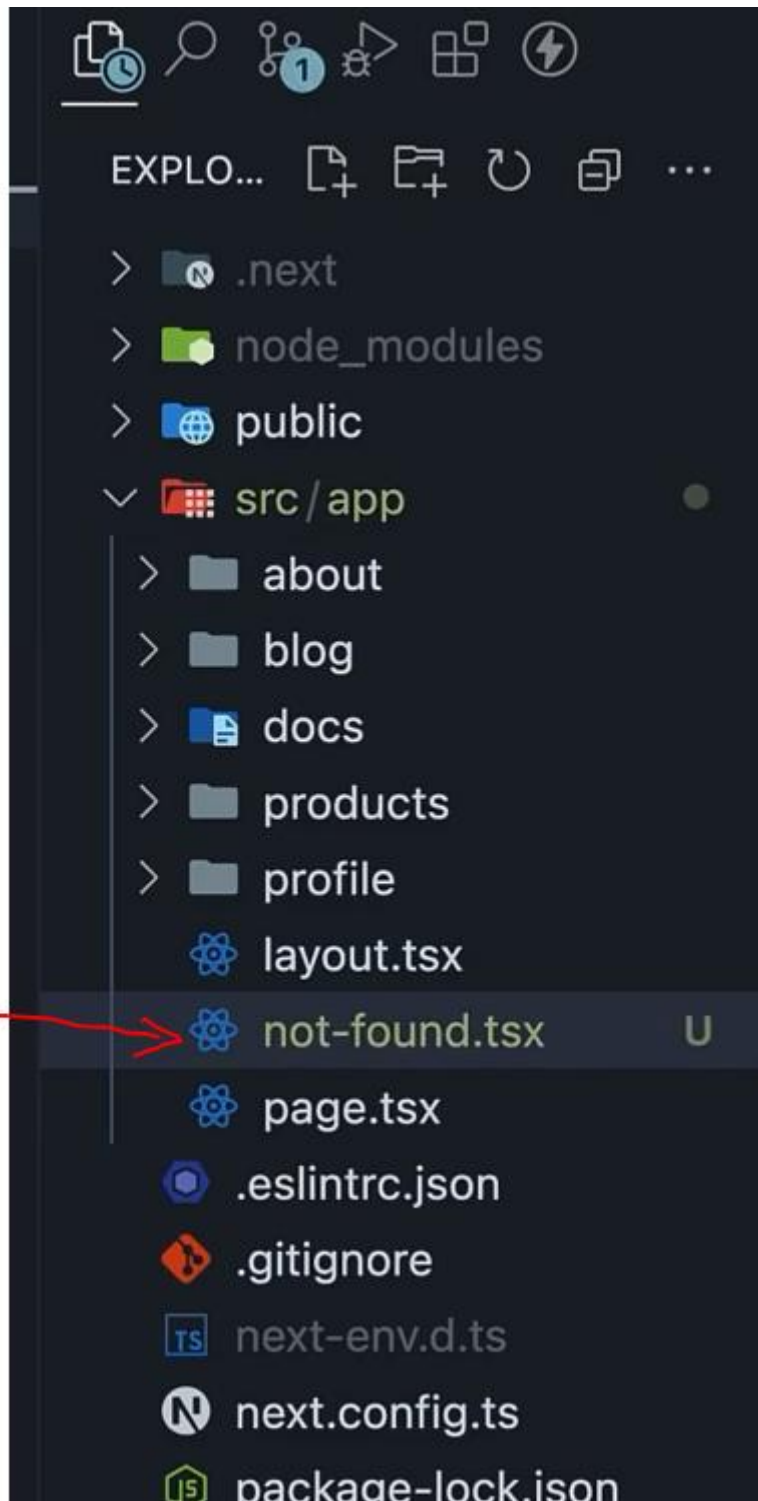


page.tsx U X

src > app > docs > [...slug] > Docs

```
1 export default async function Docs({
2   params,
3 }: {
4   params: Promise<{ slug: string[] }>;
5 }) {
6   const { slug } = await params;
7   if (slug?.length === 2) {
8     return (
9       <h1>
10         Viewing docs for feature {slug[0]} and concept {slug[1]}
11       </h1>
12     );
13   } else if (slug?.length === 1) {
14     return <h1>Viewing docs for feature {slug[0]}</h1>;
15   }
16   return <h1>Docs home page</h1>;
17 }
18
```

Not found Page



not-found.tsx U X

src > app > not-found.tsx > ...

```
1 export default function NotFound() {
2   return (
3     <div>
4       <h2>Page Not Found</h2>
5       <p>Could not find requested resource</p>
6     </div>
7   );
8 }
```

not-found.tsx .../app U not-found.tsx .../[reviewId] U X page.tsx M

src > app > products > [productId] > reviews > [reviewId] > not-found.tsx > ...

```
1 export default function NotFound() {
2   return (
3     <div>
4       <h2>Review not found</h2>
5     </div>
6   );
7 }
8
```

EXPLO... [Icons]

- > .next
- > node_modules
- > public
- > src/app
 - > about
 - > blog
 - > docs
 - > products
 - > [productId]
 - > reviews/[reviewId]
 - not-found.tsx U
 - page.tsx M
 - page.tsx
 - page.tsx
 - > profile

not-found.tsx U page.tsx M X

src > app > products > [productId] > reviews > [reviewId] > page.tsx > ProductReview

```
1 import { notFound } from "next/navigation";
2
3 export default async function ProductReview({
4   params,
5 }: {
6   params: Promise<{ productId: string; reviewId: string }>;
7 }) {
8   const { productId, reviewId } = await params;
9   if (parseInt(reviewId) > 1000) {
10    notFound();
11  }
12  return (
13    <h1>
14      Review {reviewId} for product {productId}
15    </h1>
16  );
17 }
18
```

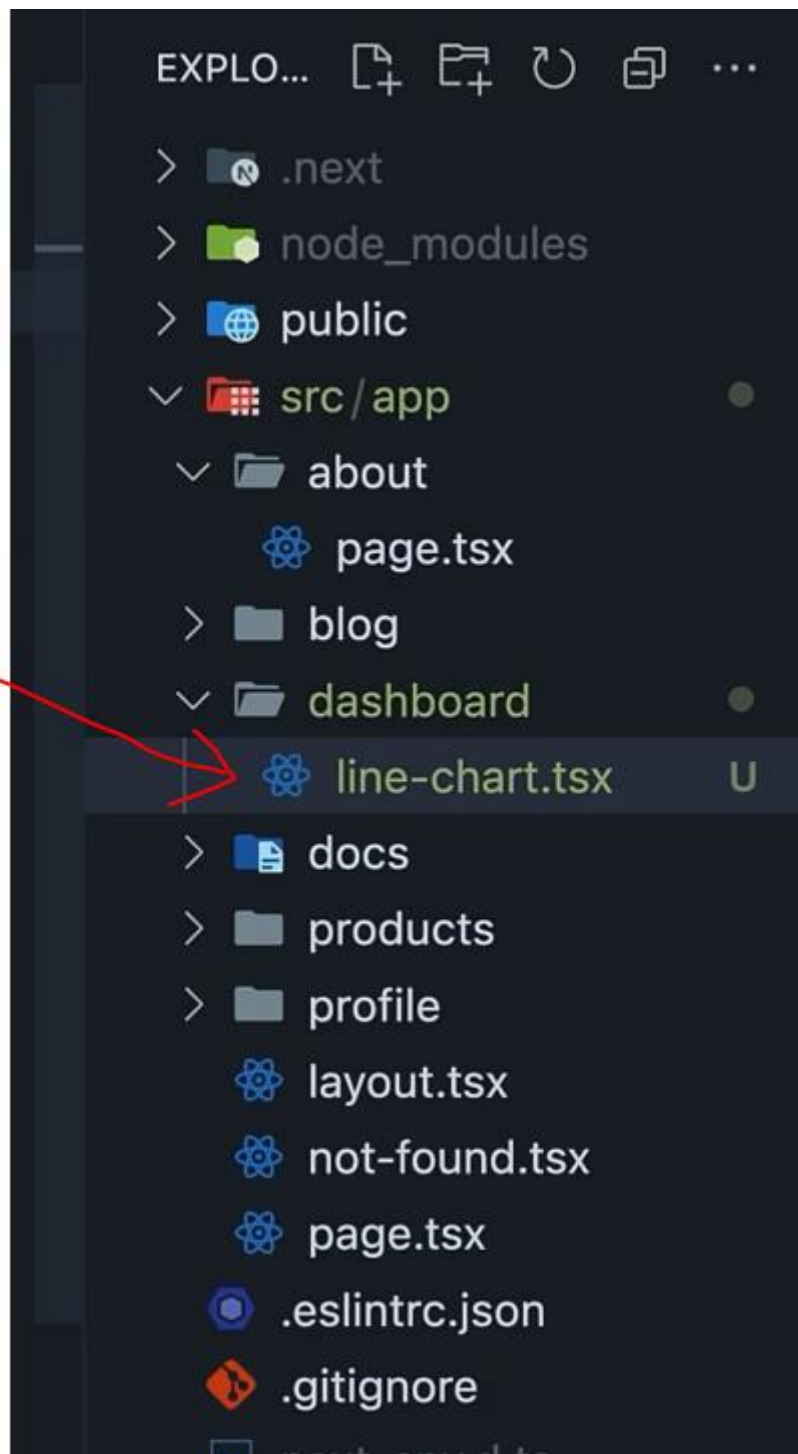
EXPLO... [Icons]

- > .next
- > node_modules
- > public
- > src/app
 - > about
 - > blog
 - > docs
 - > products
 - > [productId]
 - > reviews/[reviewId]
 - page.tsx
 - page.tsx
 - > profile
 - layout.tsx
 - not-found.tsx U
 - page.tsx
 - .eslintrc.json

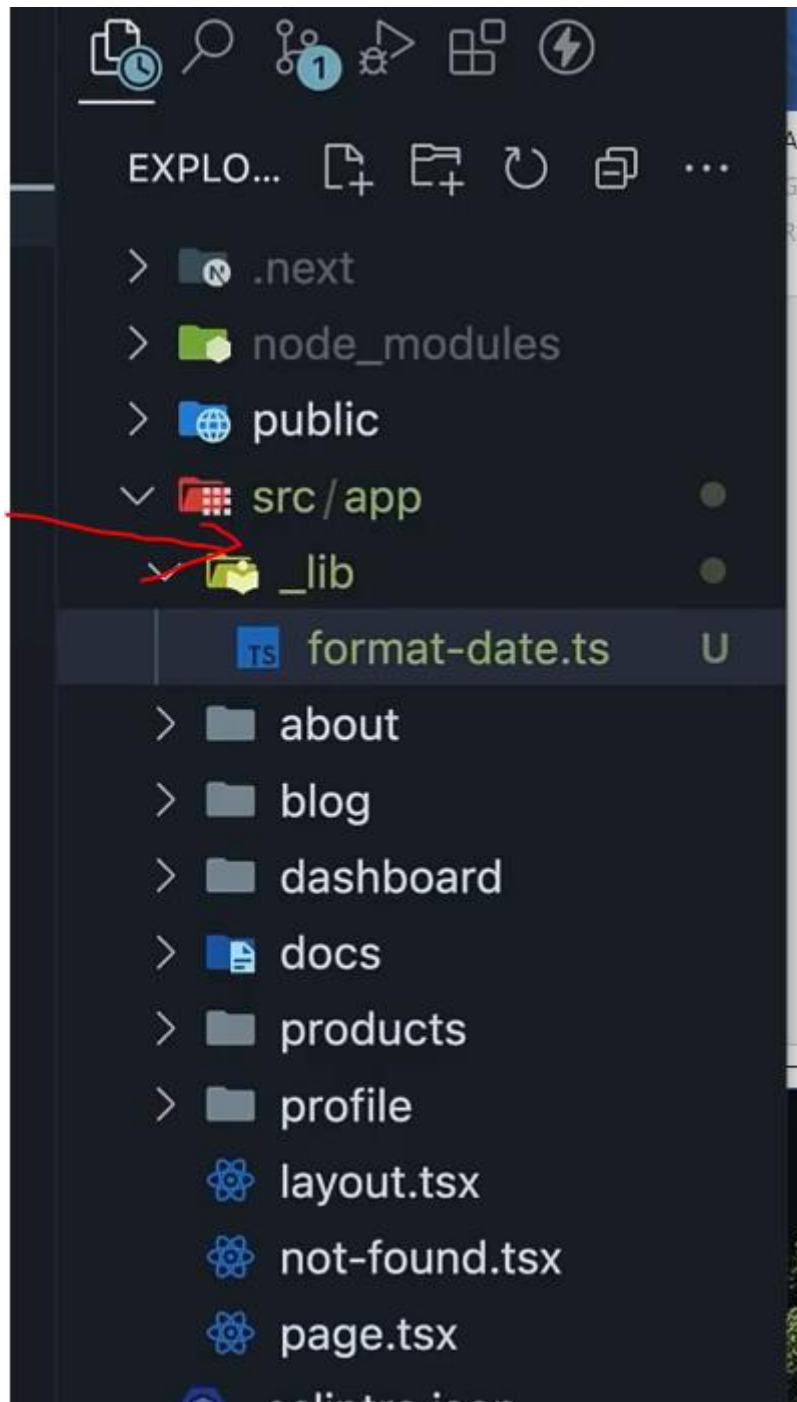
src > app > products > [productId] > reviews > [reviewId] > not-found.tsx > NotFound

```
1  "use client";
2
3  import { usePathname } from "next/navigation";
4
5  export default function NotFound() {
6    const pathname = usePathname();
7    const productId = pathname.split("/")[2];
8    const reviewId = pathname.split("/")[4];
9
10   return (
11     <div>
12       <h2>
13         Review {reviewId} not found for product {productId}
14       </h2>
15     </div>
16   );
17 }
18
```

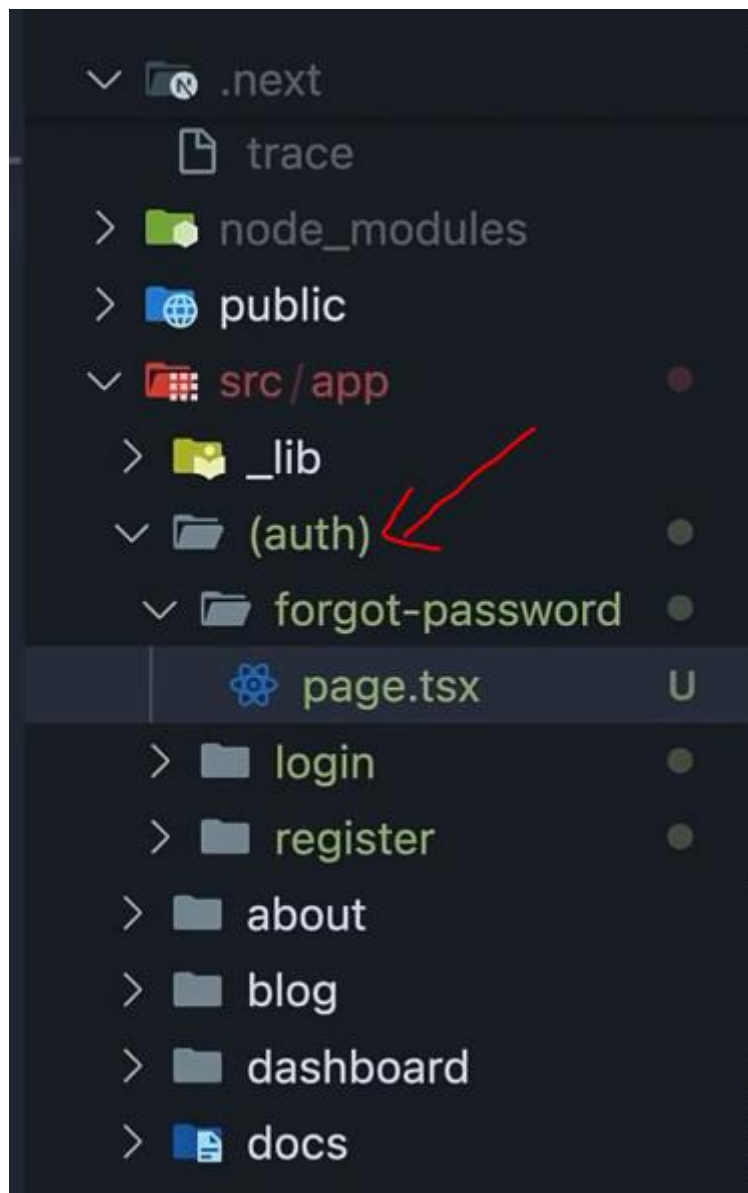
File Colocation



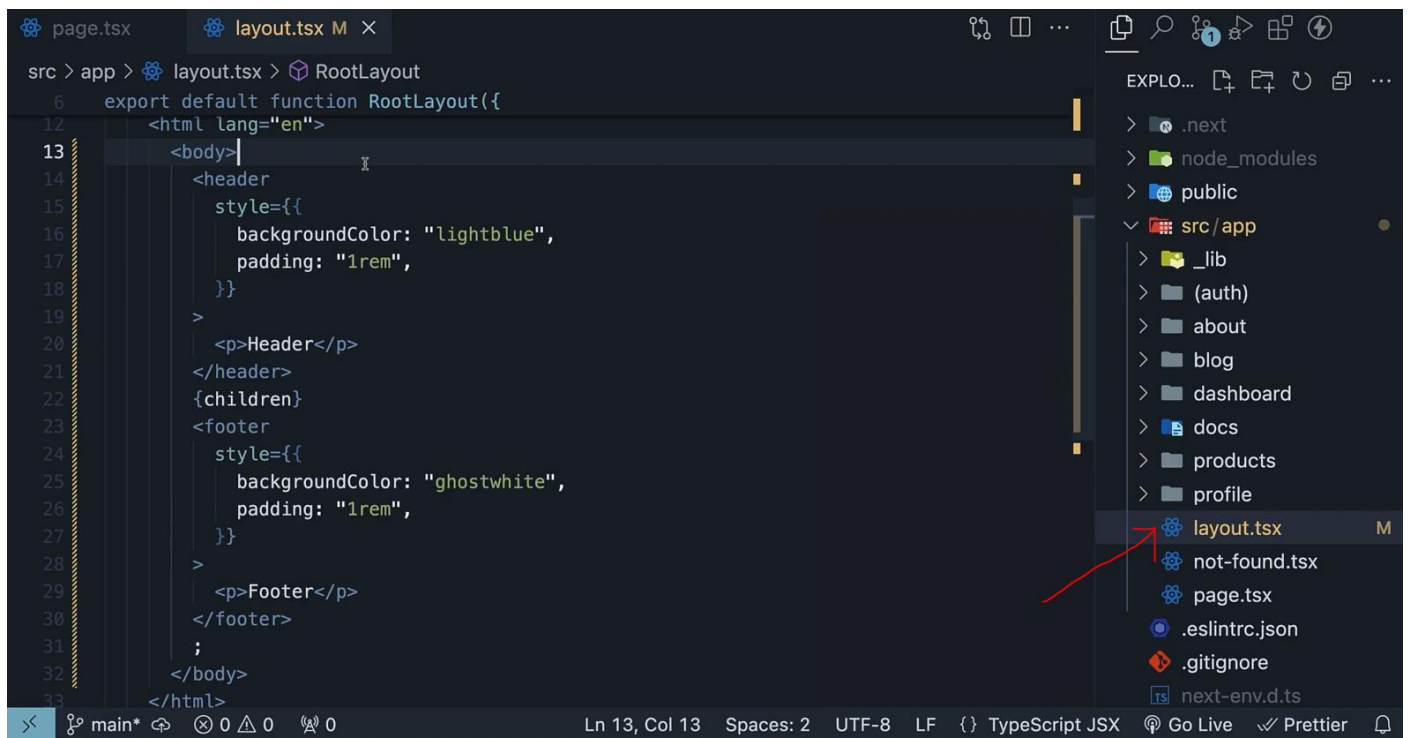
Private Folder



Route Group



Layout.tsx



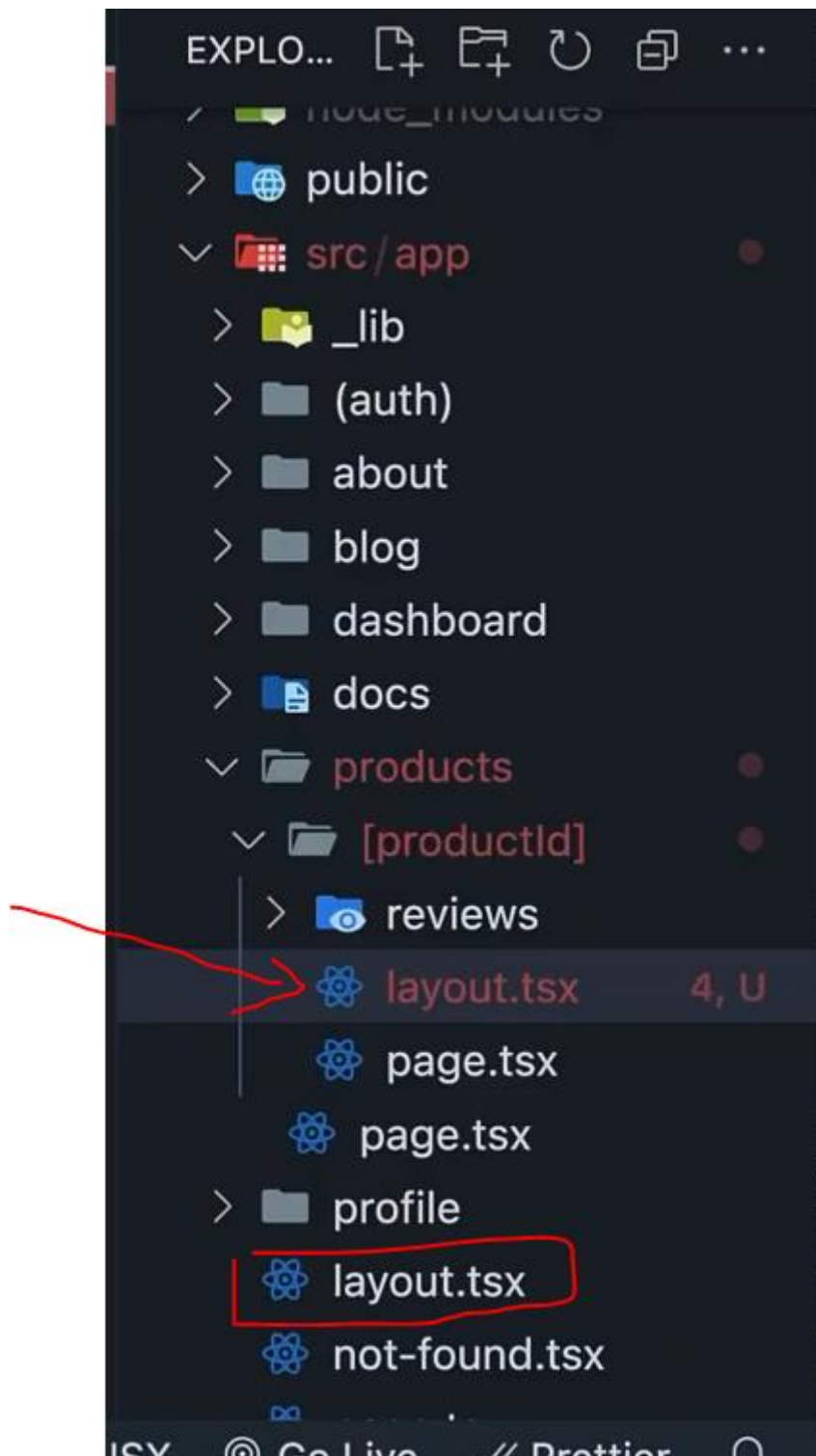
```
src > app > layout.tsx > RootLayout
6   export default function RootLayout({
12     <html lang="en">
13     <body>|
14       <header
15         style={{
16           backgroundColor: "lightblue",
17           padding: "1rem",
18         }}
19       >
20         <p>Header</p>
21       </header>
22       {children}
23     <footer
24       style={{
25         backgroundColor: "ghostwhite",
26         padding: "1rem",
27       }}
28     >
29       <p>Footer</p>
30     </footer>
31   ;
32 </body>
33 </html>
```

EXPLO...     ...

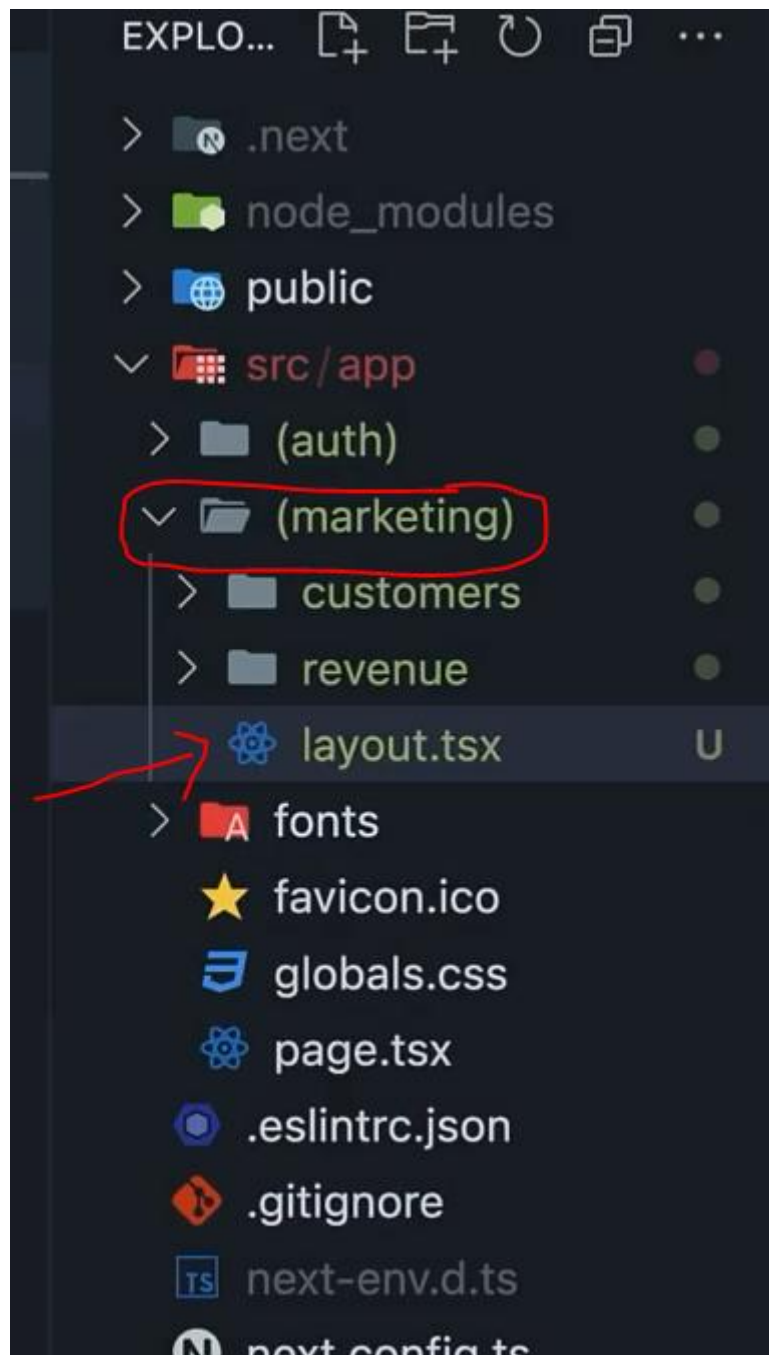
- > .next
- > node_modules
- > public
- > src/app
 - > _lib
 - > (auth)
 - > about
 - > blog
 - > dashboard
 - > docs
 - > products
 - > profile
 - layout.tsx M
 - not-found.tsx
 - page.tsx
- .eslintrc.json
- .gitignore
- next-env.d.ts

Ln 13, Col 13 Spaces: 2 UTF-8 LF {} TypeScript JSX Go Live Prettier

Nested Layout



Multiple Layout



MetaData

src > app >  layout.tsx > ...

```
1 export const metadata = {
2   title: "Next.js",
3   description: "Generated by Next.js",
4 };
5
6 export default function RootLayout({
7   children,
8 }: {
9   children: React.ReactNode;
10 }) {
11   return (
12     <html lang="en">
13       <body>
14         <header
15           style={{
16             backgroundColor: "lightblue",
17             padding: "1rem",
18           }}
19         >
20           <p>Header</p>
21         </header>
22         {children}
```

```
export const metadata = {
  title: "About Codevolution",
};

export default function About() {
  return <h1>About me</h1>;
}
```

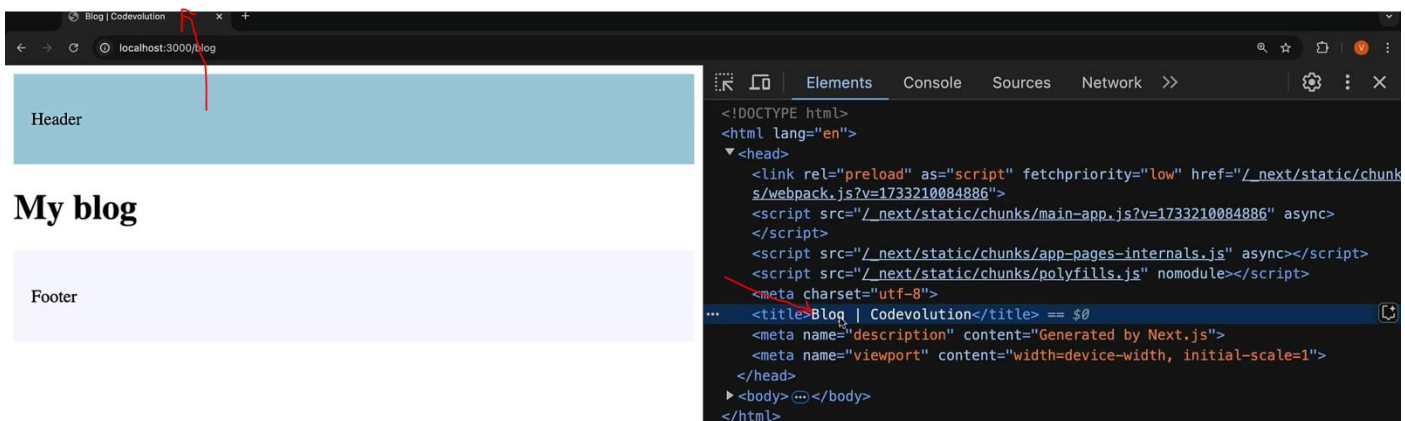
- > .next
- > node_modules
- > public
- > src/app
 - > _lib
 - > (auth)
 - > about
 - >  page.tsx M
 - > blog
 - > dashboard
 - > docs
 - > products

Title Metada Template

```
src > app > layout.tsx > metadata > title
1 import { Metadata } from "next";
2
3 export const metadata: Metadata = {
4   title: {
5     default: "Next.js Tutorial - Codevolution",
6     template: "%s | Codevolution",
7     absolute: "",
8   },
9   description: "Generated by Next.js",
10 };
11
12 export default function RootLayout({
13   children,
14 }: {
15   children: React.ReactNode;
16 }) {
17   return (
18     <html lang="en">
```

```
src > app > blog > page.tsx > metadata > title
1 import { Metadata } from "next";
2
3 export const metadata: Metadata = {
4   title: "Blog",
5 };
6
7 export default function Blog() {
8   return <h1>My blog</h1>;
9 }
10
```

Output



Link Component

```

import Link from "next/link";

export default function ProductList() {
  return (
    <>
      <Link href="/">Home</Link>
      <h1>Product list</h1>
      <h2>
        <Link href="/products/1">Product 1</Link>
      </h2>
      <h2>
        <Link href="/products/2">Product 2</Link>
      </h2>
      <h2>
        <Link href="/products/3">Product 3</Link>
      </h2>
    </>
  );
}

```

The screenshot shows a code editor with two tabs: 'page.tsx .../app M' and 'page.tsx .../products M'. The active tab is 'page.tsx .../products M', which contains the following code:

```

src > app > products > page.tsx > ProductList
3  export default function ProductList() {
4    const productId = 100;
5    return (
6      <>
7        <Link href="/">Home</Link>
8        <h1>Product list</h1>
9        <h2>
10         <Link href="/products/1">Product 1</Link>
11        </h2>
12        <h2>
13         <Link href="/products/2">Product 2</Link>
14        </h2>
15        <h2>
16         <Link href="/products/3" replace>
17           Product 3
18         </Link>
19        </h2>
20        <h2>
21         <Link href={` /products/${productId}`}>Product {productId}</Link>
22        </h2>
23      </>
24    );
25  }

```

On line 16, the word 'replace' is highlighted in blue. Two red arrows point to the 'replace' text: one from the left and one from the right. The right arrow is positioned near the closing tag of the previous link on line 15. The left arrow is positioned near the opening tag of the current link on line 16.

The right sidebar shows the 'EXPLORER' view with the following file structure:

- EXPLO...
- > .next
- > node_modules
- > public
- > src/app
 - > _lib
 - > (auth)
 - > about
 - > blog
 - > counter
 - > dashboard
 - > docs
 - > products
 - > [productId]
 - page.tsx M
 - > profile
 - layout.tsx
 - not-found.tsx

Active Link

src > app > (auth) > layout.tsx > AuthLayout > navLinks.map() callback

```
1 import Link from "next/link";
2
3 const navLinks = [
4   { name: "Register", href: "/register" },
5   { name: "Login", href: "/login" },
6   { name: "Forgot Password", href: "/forgot-password" },
7 ];
8
9 export default function AuthLayout({
10   children,
11 }): {
12   children: React.ReactNode;
13 } {
14   return (
15     <div>
16       {navLinks.map((link) => {
17         return (
18           <Link href={link.href} key={link.name}>
19             {link.name}
20           </Link>
21         );
22       })}
23     </div>
24   );
25 }
```

EXPLO...    ...

- > .next
- > node_modules
- > public
- > src/app
 - > _lib
 - > (auth)
 - > forgot-password
 - > login
 - > register
 - layout.tsx U
- > about
- > blog
- > counter
- > dashboard
- > docs
- > products
- > profile
- layout.tsx

src > app > (auth) > layout.tsx > ...

```
1 "use client";
2
3 import Link from "next/link";
4 import { usePathname } from "next/navigation";
5
6 const navLinks = [
7   { name: "Register", href: "/register" },
8   { name: "Login", href: "/login" },
9   { name: "Forgot Password", href: "/forgot-password" },
10 ];
11
12 export default function AuthLayout({
13   children,
14 }): {
15   children: React.ReactNode;
16 } {
17   const pathname = usePathname();
18   return (
19     <div>
```

```

const pathname = usePathname();
return (
  <div>
    {navLinks.map((link) => {
      const isActive =
        pathname === link.href ||
        (pathname.startsWith(link.href) && link.href !== "/");
      return (
        <Link
          className={isActive ? "font-bold mr-4" : "text-blue-500 mr-4"}
          href={link.href}
          key={link.name}
        >
          {link.name}
        </Link>
      )
    })}
  </div>
)

```

Param and Search Params

```

src > app > page.tsx > Home
1  import Link from "next/link";
2
3  export default function Home() {
4    return (
5      <>
6        <h1>Welcome home!</h1>
7        <Link href="/blog">Blog</Link>
8        <Link href="/products">Products</Link>
9        <Link href="/articles/breaking-news-123?lang=en">Read in English</Link>
10       <Link href="/articles/breaking-news-123?lang=fr">Read in French</Link>
11     </>
12   );
13 }
14

```

Param

Search param

EXPLO... [Icons]

- > node_modules
- > public
- ▼ src/app
 - > _lib
 - > (auth)
 - > about
 - > blog
 - > counter
 - > dashboard
 - > docs
 - > products
 - > profile
 - ⚙ layout.tsx
 - ⚙ not-found.tsx
 - ⚙ page.tsx M
 - ⚙ eslintignore

```
src > app > articles > [articleId] > ⚙️ page.tsx > 📦 NewsArticle
1  import Link from "next/link";
2
3  export default function NewsArticle({
4    params,
5    searchParams,
6  }: {
7    ⚠️ params: Promise<{ articleId: string }>;
8    searchParams: Promise<{ lang?: "en" | "es" | "fr" }>;
9  }) {
10   return (
11     <div>
12       <h1>News article id</h1>
13       <p>Reading in language</p>
14
15       <div>
16         <Link href="/articles/id?lang=en">English</Link>
17         <Link href="/articles/id?lang=es">Spanish</Link>
18         <Link href="/articles/id?lang=fr">French</Link>
19       </div>
20     </div>
```

```
src > app > articles > [articleId] > ⚙️ page.tsx > 📦 NewsArticle
3  export default async function NewsArticle({
4    searchParams,
5  }: {
6  }) {
7    params: Promise<{ articleId: string }>;
8    searchParams: Promise<{ lang?: "en" | "es" | "fr" }>;
9  }) {
10   const { articleId } = await params;
11   const { lang = "en" } = await searchParams;
12   return (
13     <div>
14       <h1>News article {articleId}</h1>
15       <p>Reading in {lang}</p>
16
17       <div>
18         <Link href={` /articles/${articleId}?lang=en`} >English</Link>
19         <Link href={` /articles/${articleId}?lang=es`} >Spanish</Link>
20         <Link href={` /articles/${articleId}?lang=fr`} >French</Link>
21       </div>
22     </div>
23   );
24 }
```

Navigating programmatically

```
src > app > order-product > page.tsx > OrderProduct
1 import { useRouter } from "next/navigation";
2
3 export default function OrderProduct() {
4   const router = useRouter();
5   const handleClick = () => {
6     console.log("Placing your order");
7     router.push("/");
8   };
9   return (
10     <div>
11       <h1>Order product</h1>
12       <button onClick={handleClick}>Place order</button>
13     </div>
14   );
15
16
```

EXPLO... [Icons]

- src / app
 - counter
 - dashboard
 - docs
 - order-product
 - page.tsx
 - products
 - profile
- layout.tsx
- not-found.tsx
- page.tsx
- .eslintrc.json
- .gitignore
- next-env.d.ts
- next.config.ts

```
src > app > order-product > page.tsx > OrderProduct > handleClick
1 "use client";
2 import { useRouter } from "next/navigation";
3
4 export default function OrderProduct() {
5   const router = useRouter();
6   const handleClick = () => {
7     console.log("Placing your order");
8     router.;
9   };
10   return (
11     <div>
12       <h1>0
13       <butt
14     </div>
15   );
16 }
17
```

back
forward
prefetch
push
refresh
replace

Template.tsx

```
src > app > (auth) > template.tsx > AuthLayout
13
14 export default function AuthLayout({
15   children,
16 }: {
17   children: React.ReactNode;
18 }) {
19   const [input, setInput] = useState("");
20   const pathname = usePathname();
21   return (
22     <div>
23       <div>
24         <input value={input} onChange={(e) => setInput(e.target.value)} />
25       </div>
26       {navLinks.map((link) => {
27         const isActive =
28           pathname === link.href ||
29           (pathname.startsWith(link.href) && link.href !== "/");
30         return (
31           <Link
32             className={isActive ? "font-bold mr-4" : "text-blue-500 mr-4"}
33             href={link.href}
34             key={link.name}
35           >
```

Good for Animation

EXPLO... ...

- > node_modules
- > public
- > src/app
 - > _lib
 - > (auth)
 - > forgot-password
 - > login
 - > register
 - > styles.css
 - > template.tsx
 - > about
 - > articles
 - > blog
 - > counter
 - > dashboard
 - > docs
 - > order-product

Loading UI

```
src > app > blog > loading.tsx > ...
1 export default function Loading() {
2   return <h1>Loading</h1>;
3 }
4
```

EXPLO... ...

- > .next
- > node_modules
- > public
- > src/app
 - > _lib
 - > (auth)
 - > about
 - > articles
 - > blog
 - > first
 - > second
 - > loading.tsx
 - > page.tsx
 - > counter
 - > dashboard
 - > docs

```
src > app > blog > page.tsx > Blog > <function> > setTimeout() callback
1 import { Metadata } from "next";
2
3 export const metadata: Metadata = {
4   title: {
5     absolute: "Blog",
6   },
7 };
8
9 export default async function Blog() {
10   await new Promise((resolve) => {
11     setTimeout(() => {
12       resolve("intentional delay");
13     }, 2000);
14   });
15   return <h1>My blog</h1>;
16 }
17
```

EXPLO... ...

- > .next
- > node_modules
- > public
- > src/app
 - > _lib
 - > (auth)
 - > about
 - > articles
 - > blog
 - > first
 - > second
 - > loading.tsx
 - > page.tsx
 - > counter
 - > dashboard

Error UI

src > app > products > [productId] > reviews > [reviewId] > page.tsx > ProductReview

```
1 import { notFound } from "next/navigation";
2
3 function getRandomInt(count: number) {
4   return Math.floor(Math.random() * count);
5 }
6
7 export default async function ProductReview({
8   params,
9 }: {
10   params: Promise<{ productId: string; reviewId: string }>;
11 }) {
12   const random = getRandomInt(2);
13   if (random === 1) {
14     throw new Error("Error loading review");
15   }
16   const { productId, reviewId } = await params;
17   if (parseInt(reviewId) > 1000) {
18     notFound();
19   }
20   return (
21     <h1>
22     Review {reviewId} for product {productId}
```

Intentional
Error

EXPLO... [icons]

src/app

- > (auth)
- > about
- > articles
- > blog
- > counter
- > dashboard
- > docs
- > order-product
- > products
 - > [productId]
 - > reviews/[reviewId]
 - not-found.tsx
 - page.tsx M
 - layout.tsx
 - page.tsx
 - page.tsx

page.tsx M error.tsx U X

src > app > products > [productId] > reviews > [reviewId] > error.tsx > ErrorBoundary

```
1 "use client";
2
3 export default function ErrorBoundary() {
4   return <div>Error in reviewId</div>;
5 }
6
```

EXPLO... [icons]

- src/app
 - articles
 - blog
 - counter
 - dashboard
 - docs
 - order-product
 - products
 - [productId]
 - reviews/[reviewId]
 - error.tsx U
 - not-found.tsx
 - page.tsx M
 - layout.tsx

page.tsx M error.tsx U X

src > app > products > [productId] > reviews > [reviewId] > error.tsx > ErrorBoundary

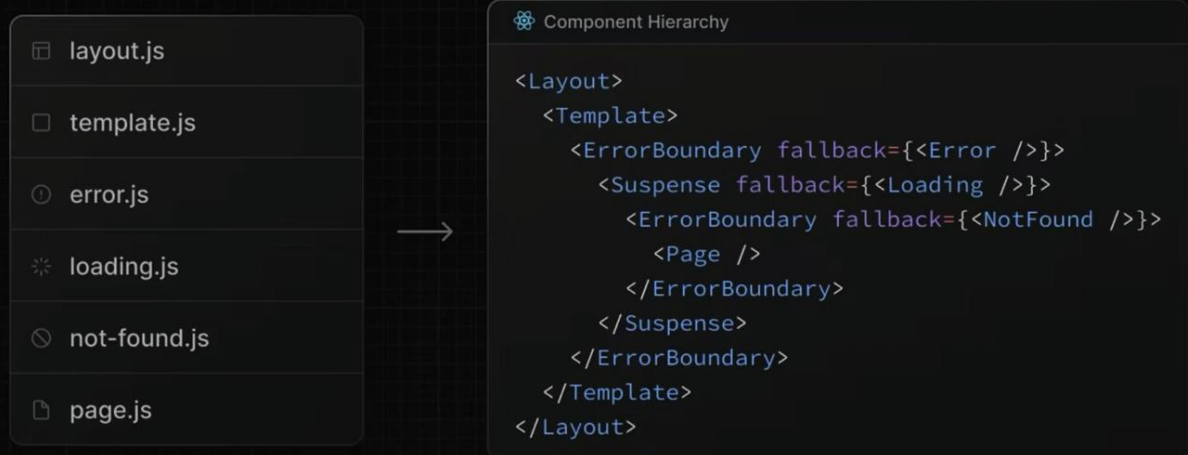
```
1 "use client";
2
3 export default function ErrorBoundary({ error }: { error: Error }) {
4   return <div>{error.message}</div>;
5 }
6
```

show
specific
error

EXPLO... [icons]

- src/app
 - articles
 - blog
 - counter
 - dashboard
 - docs
 - order-product
 - products
 - [productId]
 - reviews/[reviewId]
 - error.tsx U
 - not-found.tsx
 - page.tsx M
 - layout.tsx

Component hierarchy



Reset from error

```
page.tsx  error.tsx M X
src > app > products > [productId] > reviews > [reviewId] > error.tsx > ErrorBoundary
1  "use client";
2
3  export default function ErrorBoundary({
4    error,
5    reset,
6  }: {
7    error: Error;
8    reset: () => void;
9  }) {
10   return (
11     <div>
12       <p>{error.message}</p>
13       <button onClick={() => reset()}>Try again</button>
14     </div>
15   );
16 }
17
```

Recover using ueRouter

```

src > app > products > [productId] > reviews > [reviewId] > error.tsx > ErrorBoundary
3 import { useRouter } from 'next/navigation';
4 import { startTransition } from "react";
5
6 export default function ErrorBoundary({
7   error,
8   reset,
9 }: {
10   error: Error;
11   reset: () => void;
12 }) {
13   const router = useRouter();
14   const reload = () => {
15     startTransition(() => {
16       router.refresh();
17       reset();
18     });
19   };
20   return (
21     <div>
22       <p>{error.message}</p>
23       <button onClick={() => reload()}>Try again</button>
24     </div>

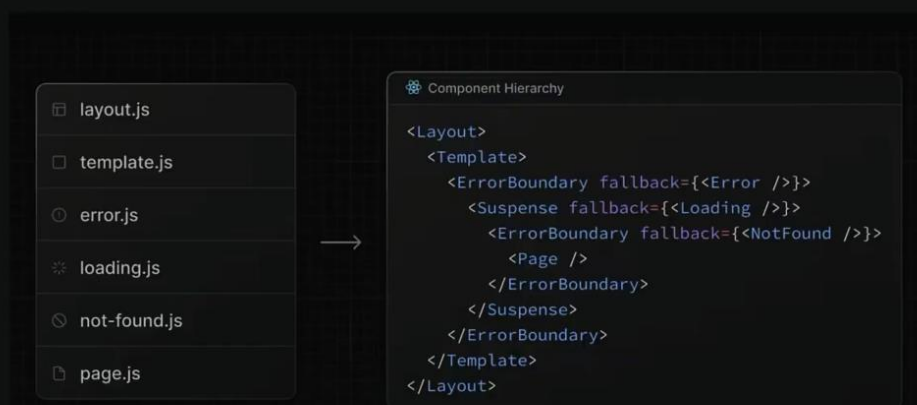
```

update onclick handler from reset() to reload()

Handling errors in layouts

The error boundary won't catch errors thrown in layout.tsx within the same segment because of how the component hierarchy works

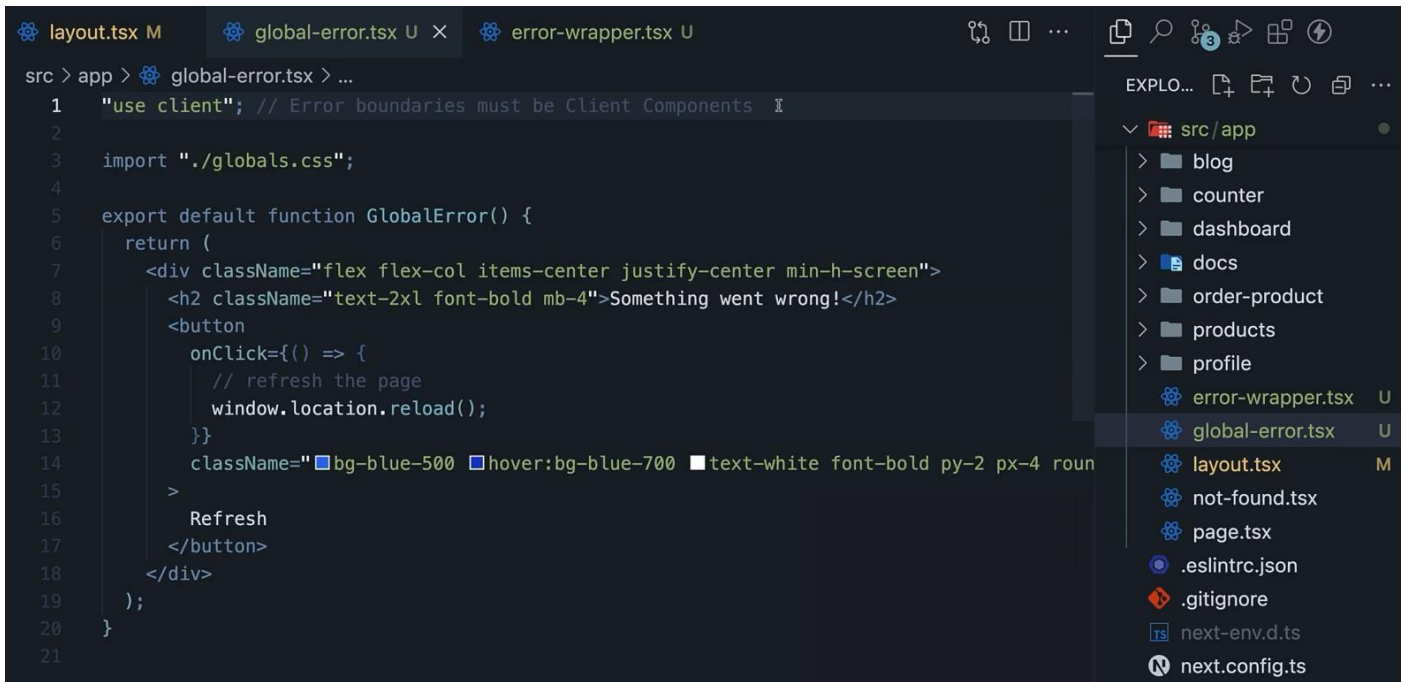
The layout actually sits above the error boundary in the component tree



Error do not catch from same level layout.tsx , because of layout sits above the ErrorBoundry, so need to place error.tsx one level above

Global errors

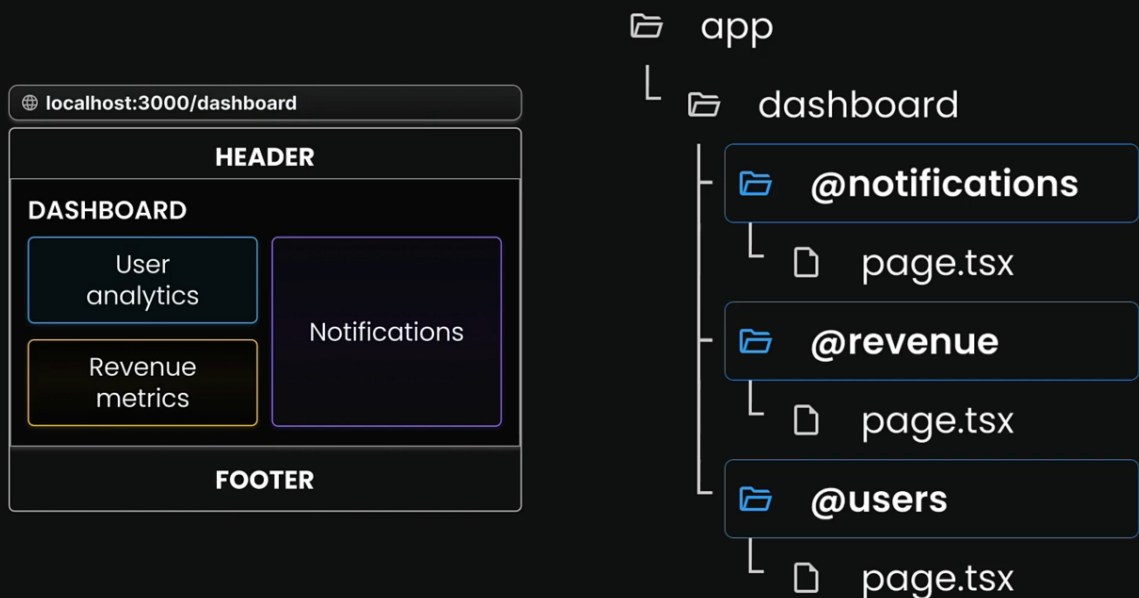
```
layout.tsx M  global-error.tsx U  error-wrapper.tsx U
src > app > global-error.tsx > ...
1  "use client"; // Error boundaries must be Client Components
2
3  import "../globals.css";
4
5  export default function GlobalError() {
6    return (
7      <div className="flex flex-col items-center justify-center min-h-screen">
8        <h2 className="text-2xl font-bold mb-4">Something went wrong!</h2>
9        <button
10          onClick={() => {
11            // refresh the page
12            window.location.reload();
13          }}
14          className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded"
15        >
16          Refresh
17        </button>
18      </div>
19    );
20  }
21
```

A screenshot of a code editor with a dark theme. The main editor shows the content of 'global-error.tsx'. The file explorer on the right shows the project structure, with 'global-error.tsx' highlighted. The breadcrumb at the top of the editor indicates the path 'src > app > global-error.tsx'.

global-error.tsx will catch error from root layout

Parallel routes

Parallel routes contd.



Unmatched Route

Unmatched routes

Navigation from the UI

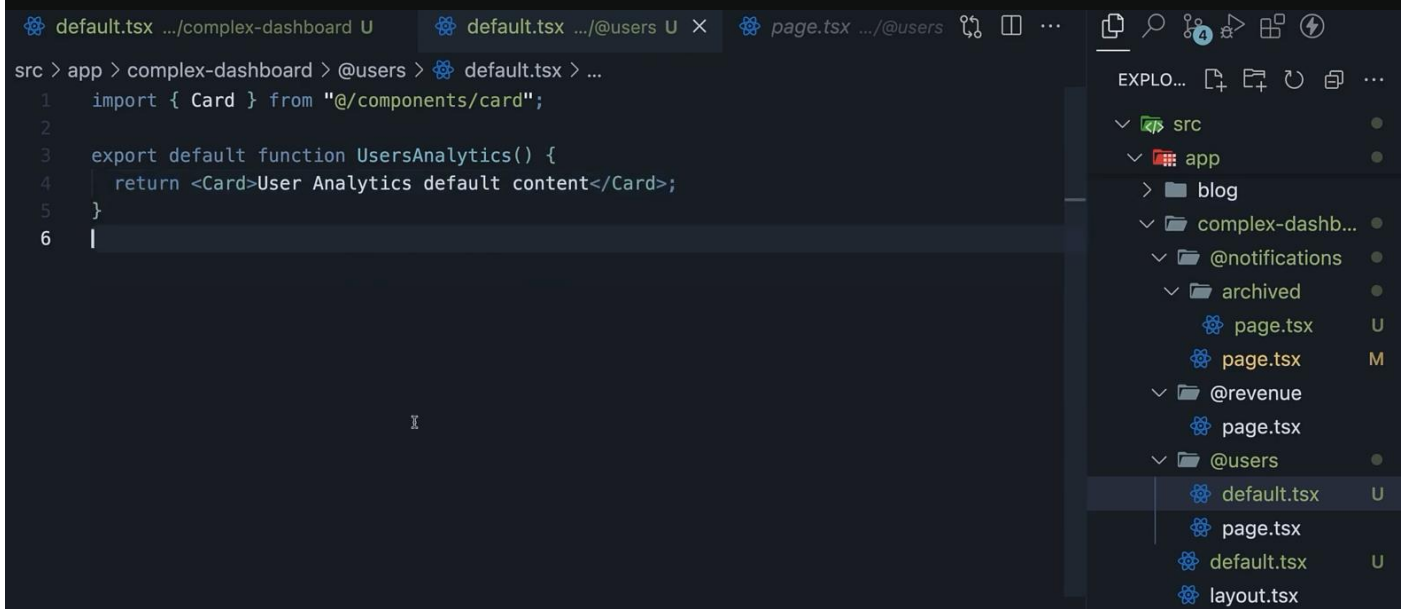
When navigating through the UI (like clicking links), Next.js keeps showing whatever was in the unmatched slots before

Page reload

Next.js looks for a `default.tsx` file in each unmatched slot

This file is critical as it serves as a fallback to render content when the framework cannot retrieve a slot's active state from the current URL

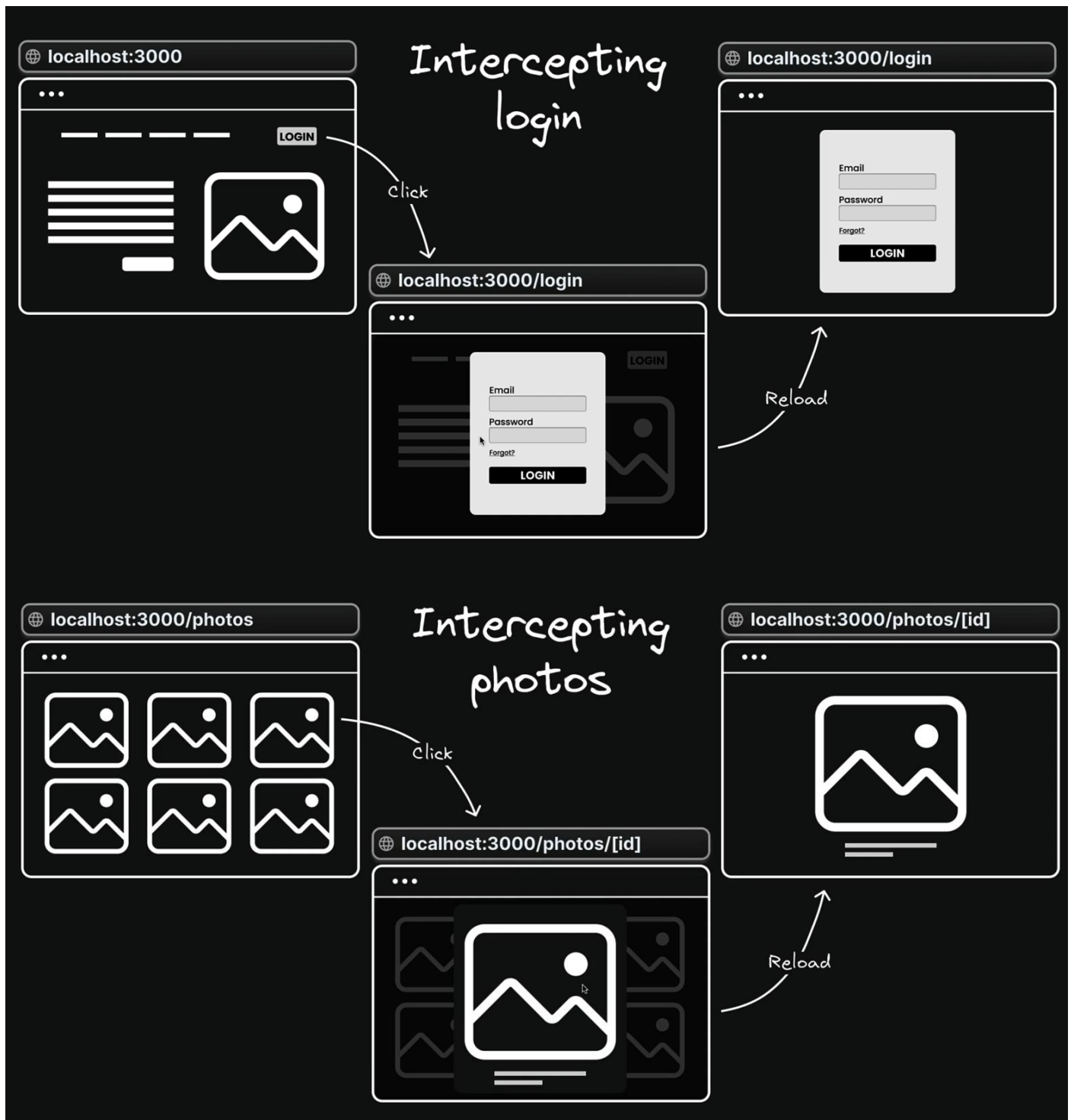
Without the file, you'll get a 404 error



So, if we create a route inside parallel route then on clicking link no change occur in unmatched route ,only change the matched slot , but when reload then unmatched route show 404 error .So, we need to put default.tsx in each of the file

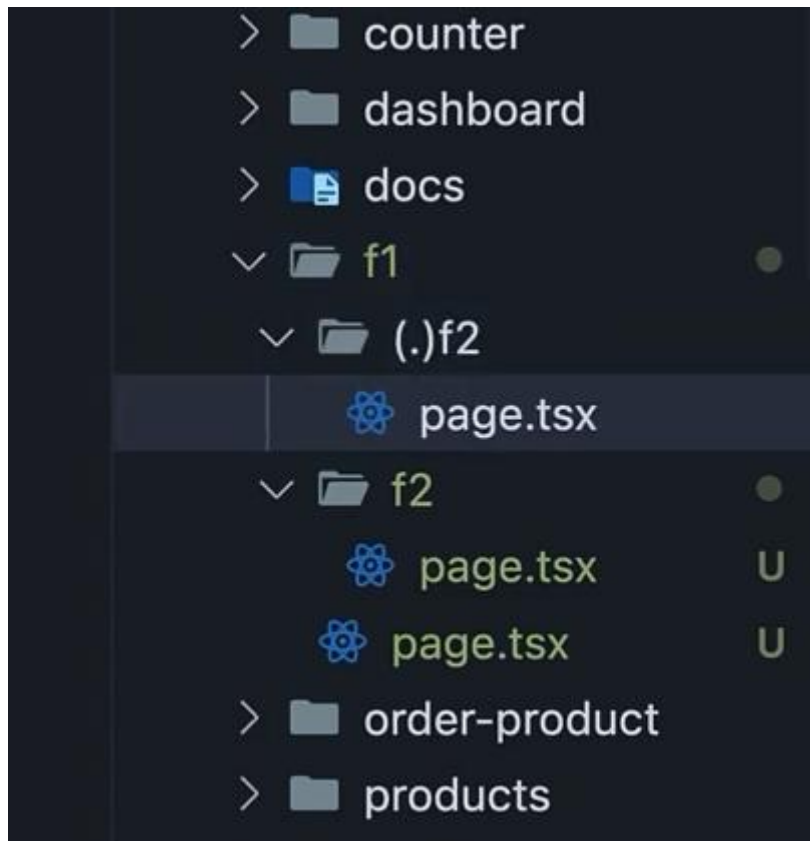
Parallel route can be used as a **conditional route**

Intercepted Route



We consider two think source and the target

Suppose we are navigating from gallery page to singlePhoto page, so gallery page is source and single photo page is the target



So, now when move to 1st page 2nd page then, first go through intercepted routes. If we reload then comes from actual routes

(.)f1-> same level

(..)f1-> one level above

(..)(..)f1 -> two level above

(...)f1 -> match from root app directory

Parallel intercepted route

1. Create layout.tsx in photo-feed folder. It will get children and modal props and show side by side
`{modal} {children}`

2. Inside @modal parallel route create default.tsx . so that in first loading , {modal} return null and only page.tsx load
3. Create page.tsx where phot gallery should be loaded
4. Create [id] dynamic route inside photo-feed and create page.tsx inside to make a single photo page..
5. Create a intercepted route inside @modal folder so that (.)[id] intercept same level [id] route...

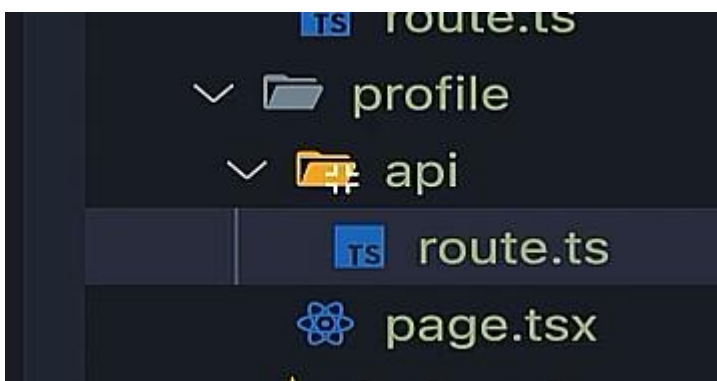
Route Handler



```
route.ts U X
src > app > hello > route.ts > GET
1 export async function GET() {
2   return new Response("Hello world!");
3 }
4
```

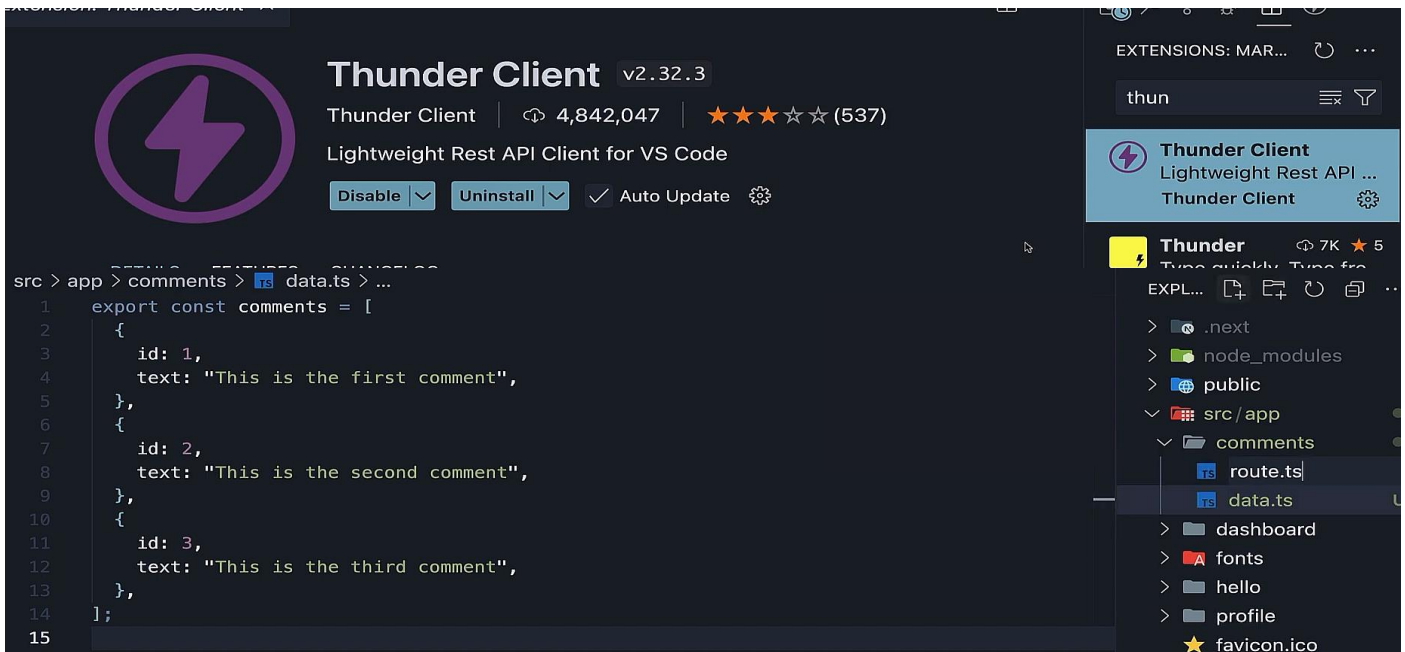
The screenshot shows a code editor with a file named 'route.ts' open. The code defines an asynchronous GET function that returns a new Response with the text 'Hello world!'. The file explorer on the right shows the project structure, including 'src/app', 'hello', and 'route.ts'.

Standard javascript response



route.ts and page.tsx can stay together but route get priority. So move route.ts to api subfolder

Handling Get request



Post request

```
POST http://localhost:3000/comments Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

1 {
2   "text": "New comm"
3 }

src > app > comments > route.ts > POST
1 import { comments } from "../data";
2
3 export async function GET() {
4   return Response.json(comments);
5 }
6
7 export async function POST(request: Request) {
8   const comment = await request.json();
9   const newComment = {
10     id: comments.length + 1,
11     text: comment.text,
12   };
13   comments.push(newComment);
14   return new Response(JSON.stringify(newComment), {
15     headers: { "Content-Type": "application/json" },
16     status: 201,
17   });
18 }
```

Dynamic Route Handler



```
src > app > comments > [id] > route.ts > ...
1 import { comments } from "../../data";
2
3 export async function GET(
4   _request: Request,
5   { params }: { params: Promise<{ id: string }> }
6 ) {
7   const { id } = await params;
8   const comment = comments.find((comment) => comment.id === parseInt(id));
9   return Response.json(comment);
10 }
```

EXPL... [Icons]

- > .next
- > node_modules
- > public
- > src/app
 - > comments
 - > [id]
 - route.ts
 - data.ts

Handling Patch Request

The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** http://localhost:3000/comments/3
- Tab:** Body
- Format:** JSON
- Body Content:**

```
1 {  
2   "text": "Updated comment"  
3 }
```

```
export async function PATCH(  
  request: Request,  
  { params }: { params: Promise<{ id: string }> }  
) {  
  const { id } = await params;  
  const body = await request.json();  
  const { text } = body;  
  
  const index = comments.findIndex((comment) => comment.id === parseInt(id));  
  comments[index].text = text;  
  return Response.json(comments[index]);  
}
```

Handling Delete Request

The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** http://localhost:3000/comments/3
- Tab:** Body
- Format:** JSON
- Body Content:**

```
1 |
```

```
export async function DELETE(  
  _request: Request,  
  { params }: { params: Promise<{ id: string }> }  
) {  
  const { id } = await params;  
  const index = comments.findIndex((comment) => comment.id === parseInt(id));  
  const deletedComment = comments[index];  
  comments.splice(index, 1);  
  return Response.json(deletedComment);  
}
```

Url Query Parameter

localhost:3000/comments?query=first

```
import { type NextRequest } from "next/server";  
import { comments } from "../data";  
  
export async function GET(request: NextRequest) {  
  return Response.json(comments);  
}
```

```
export async function GET(request: NextRequest) {  
  const searchParams = request.nextUrl.searchParams;  
  const query = searchParams.get("query");  
  const filteredComments = query  
    ? comments.filter((comment) => comment.text.includes(query))  
    : comments;  
  return Response.json(filteredComments);  
}
```

Headers in route Handler

Request headers

'User-Agent' which

'Accept' which is
the client can pr

'Authorization'

Response header **'Content-Type'**

How to grab headers

```
import { type NextRequest } from "next/server";

export async function GET(request: NextRequest) {
  const requestHeaders = new Headers(request.headers);
  console.log(requestHeaders.get("Authorization"));
  return new Response("Profile API data");
}
```

Another method, this is good

```
import { type NextRequest } from "next/server";
import { headers } from "next/headers";

export async function GET(request: NextRequest) {
  // const requestHeaders = new Headers(request.headers);
  // console.log(requestHeaders.get("Authorization"));

  const headersList = await headers();
  console.log(headersList.get("Authorization"));
  return new Response("Profile API data");
}
```

Set response header

```
return new Response("<h1>Profile API data</h1>", {
  headers: {
    "Content-Type": "text/html",
  },
});
```

Cookies in web browser

```
return new Response("<h1>Profile API data</h1>", {  
  headers: {  
    "Content-Type": "text/html",  
    "Set-Cookie": "theme=dark",  
  },  
});
```

One way to set cookie is set to header like above

```
const theme = request.cookies.get("theme");  
console.log(theme);
```

This is how to print cookie

Second way

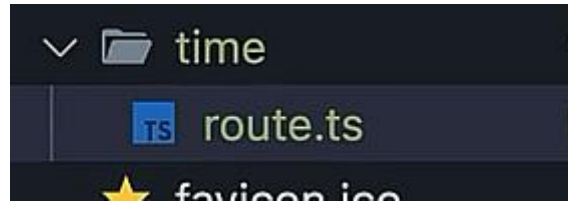
```
import { headers, cookies } from "next/headers";  
  
const cookieStore = await cookies()  
cookieStore.set("resultsPerPage", "20")  
  
console.log(cookieStore.get("resultsPerPage"))
```

Redirects in route handler

```
import { redirect } from "next/navigation";
```

```
export async function GET() {  
  redirect("/api/v2/users");  
}
```

Caching in route handler



```
src > app > time > route.ts > ...
1  export async function GET() {
2    return Response.json({ time: new Date().toLocaleTimeString() });
3  }
4
```

No caching by default

```
src > app > time > route.ts > ...
1  export const dynamic = "force-static";
2
3  export async function GET() {
4    return Response.json({ time: new Date().toLocaleTimeString() });
5  }
6
```

Still no change will be occurred because there is no caching in development

Time is cached when app is built only

Incremental static generation

```
export const dynamic = "force-static";
export const revalidate = 10;

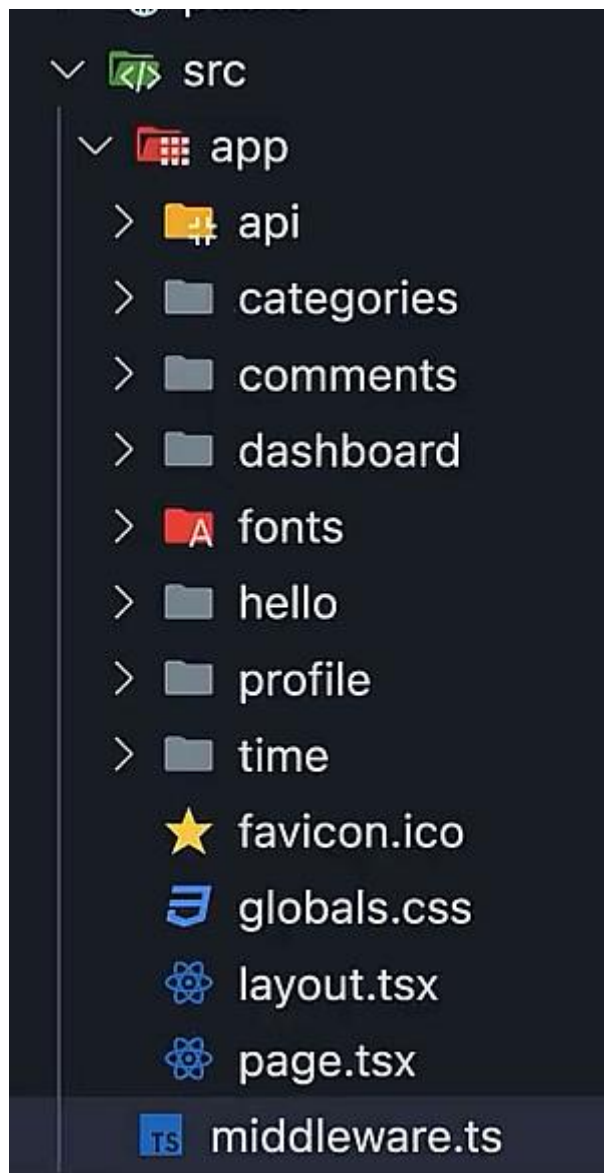
export async function GET() {
  return Response.json({ time: new Date().toLocaleTimeString() });
}
```

Other HTTP methods like POST, PUT, or DELETE are never cached

if you're using dynamic functions like *headers()* and *cookies()*, or working with the *request* object in your GET method, caching won't be applied

MiddleWare.ts

It does this at a global level, significantly enhancing features like redirects, URL rewrites, authentication, headers, cookies, and more



Create middleware.ts in src folder

Middleware lets you specify paths where it should be active

- Custom matcher config
- Conditional statements

```
src > TS middleware.ts > ...
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
4  export function middleware(request: NextRequest) {
5    return NextResponse.redirect(new URL("/", request.url));
6  }
7
8  export const config = {
9    matcher: "/profile",
10 };
```

Another approach

```
src > TS middleware.ts > ...
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
4  export function middleware(request: NextRequest) {
5    // return NextResponse.redirect(new URL("/", request.url));
6
7    if (request.nextUrl.pathname === "/profile") {
8      return NextResponse.redirect(new URL("/hello", request.nextUrl));
9    }
10 }
```

Use rewrite to change the url same only change data

```
src > TS middleware.ts > ...
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
4  export function middleware(request: NextRequest) {
5    // return NextResponse.redirect(new URL("/", request.url));
6
7    if (request.nextUrl.pathname === "/profile") {
8      return NextResponse.rewrite(new URL("/hello", request.nextUrl));
9    }
10 }
```

Cookies and Headers in middleware

```
src > TS middleware.ts > middleware
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
4  export function middleware(request: NextRequest) {
5      const response = NextResponse.next();
6      const themePreference = request.cookies.get("theme");
7      if (!themePreference) {
8          response.cookies.set("theme", "dark");
9      }
10
11     return response;
12
13     // return NextResponse.redirect(new URL("/", request.url));
14     // if (request.nextUrl.pathname === "/profile") {
15     //     return NextResponse.rewrite(new URL("/hello", request.url));
16     // }
17 }
```

Custom headers

```

src > TS middleware.ts > middleware
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
4  export function middleware(request: NextRequest) {
5      const response = NextResponse.next();
6      const themePreference = request.cookies.get("theme");
7      if (!themePreference) {
8          response.cookies.set("theme", "dark");
9      }
10
11     response.headers.set("custom-header", "custom-value");
12
13     return response;
14
15     // return NextResponse.redirect(new URL("/", request.url));
16     // if (request.nextUrl.pathname === "/profile") {
17     //     return NextResponse.rewrite(new URL("/hello", request.url));
18     // }
19 }

```

Rendering in NextJS

Client side rendering (CSR)

Server-side solutions

1. Static Site Generation (SSG)
2. Server-Side Rendering (SSR)

By default server component

“use client” for client component

Static rendering

Ssr dynamic rendering

```
src > app > about >  page.tsx >  AboutPage
1  import { cookies } from "next/headers";
2
3  export default async function AboutPage() 
4    const cookieStore = await cookies();
5    const theme = cookieStore.get("theme");
6    console.log(theme);
7    console.log("About server component");
8    return <h1>About page {new Date().toLocaleTimeString()}</h1>;
9  
10
```

generateStaticParams

```
src > app > products > [id] >  page.tsx >  ProductPage >  id
1  export default async function ProductPage({
2    params,
3  }: {
4    params: Promise<{ id: string }>;
5  }) {
6    const { id } = await params;
7    return (
8      <h1>
9        Product {id} details rendered at {new Date().toLocaleTimeString()}
10     </h1>
11   );
12 }
13
```

EXPLORER...   

- > .next
- > node_modules
- > public
- ✓ src/app
 - > about
 - > dashboard
 - ✓ products
 - ✓ [id]
 -  page.tsx
 -  page.tsx
 - ★ favicon.ico

```

export async function generateStaticParams() {
  return [{ id: "1" }, { id: "2" }, { id: "3" }];
}

export default async function ProductPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  return (
    <h1>
      Product {id} details rendered at {new Date().toLocaleTimeString()}
    </h1>
  );
}

```

For multiple slug of dynamic route

```

export async function generateStaticParams() {
  return [
    { category: "electronics", product: "smartphone" },
    { category: "electronics", product: "laptop" },
    { category: "books", product: "science-fiction" },
    { category: "books", product: "biography" },
  ];
}

```

dynamicParams contd.

true:

If you're building an e-commerce site, you'll probably want to keep dynamicParams set to true

This way, you can pre-render your most popular product pages for better performance, but still allow access to all your other products - they'll just be rendered on demand

false:

if you're working with something like a blog where you have a smaller, more fixed number of pages, you can pre-render all of them and set dynamicParams to false

If someone tries to access a blog post that doesn't exist, they'll get a clean 404 error instead of waiting for a page that will never exist

Streaming

```
export const Reviews = async () => {  
  await new Promise((resolve) => setTimeout(resolve, 4000));  
  return <div>Reviews</div>;  
};
```

```
import { Suspense } from "react";

import { Product } from "@components/product";
import { Reviews } from "@components/reviews";

export default function ProductReviews() {
  return (
    <div>
      <h1>Product reviews</h1>
      <Suspense fallback={<p>Loading product details...</p>}>
        <Product />
      </Suspense>
      <Suspense fallback={<p>Loading reviews...</p>}>
        <Reviews />
      </Suspense>
    </div>
  );
}
```

wrapping suspense will stream when each component will
be available

Only server function

```
import "server-only";

export const serverSideFunction = () => {
  console.log(
    `use multiple libraries,
      use environment variables,
      interact with a database,
      process confidential information`
  );
  return "server result";
};
```

You can import client component inside server component

But can not import server component inside client component directly

We can pass client data to server using composition pattern .. Like this <Provider client><Layout server></Provider client>

```
import "client-only";

export const clientSideFunction = () => {
  console.log(
    `use window object,
      use localStorage`
  );
  return "client result";
};
```

Only works in client only

Data Fetching

```
import { useState, | } from "react";

type User = {
  id: number;
  name: string;
  username: string;
  email: string;
  phone: string;
};

export default function UsersClient() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
}
```

```
useEffect(() => {  
  async function fetchUsers() {  
    try {  
      const response = await fetch(  
        "https://jsonplaceholder.typicode.com/users"  
      );  
      if (!response.ok) throw new Error("Failed to fetch users");  
      const data = await response.json();  
      setUsers(data);  
    } catch (err) {  
      if (err instanceof Error) {  
        setError(err.message);  
      } else {  
        setError("An unknown error occurred");  
      }  
    } finally {  
      setLoading(false);  
    }  
  }  
  fetchUsers();  
}, []);
```

```
if (loading) return <div>Loading...</div>;  
if (error) return <div>{error}</div>;
```

```

return (
  <ul className="space-y-4 p-4">
    {users.map((user) => (
      <li
        key={user.id}
        className="p-4 bg-white shadow-md rounded-lg text-gray-700"
      >
        <div className="font-bold">{user.name}</div>
        <div className="text-sm">
          <div>Username: {user.username}</div>
          <div>Email: {user.email}</div>
          <div>Phone: {user.phone}</div>
        </div>
      </li>
    ))}
  </ul>
);

```

Data Fetch using server component

```

type User = {
  id: number;
  name: string;
  username: string;
  email: string;
  phone: string;
};

export default async function UsersServer() {

```

```

  export default async function UsersServer() {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    const users: User[] = await response.json();
    console.log(users);
  }

```

Parallel Data fetching

```
const postsData = getUserPosts(id);
const albumsData = getUserAlbums(id);

const [posts, albums] = await Promise.all([postsData, albumsData]);
```

Server actions

```
async function createProduct() {
  "use server";
}

return (
  <form action={createProduct} className="p-4 space-y-4 max-w-96">
    <label className="text-white">
      async function createProduct(formData: FormData) {
```

```
const title = formData.get("title") as string;
const price = formData.get("price") as string;
const description = formData.get("description") as string;
```

```
import { addProduct } from "@prisma-db";
import { redirect } from "next/navigation";

export default function AddProductPage() {
  async function createProduct(formData: FormData) {
    "use server";

    const title = formData.get("title") as string;
    const price = formData.get("price") as string;
    const description = formData.get("description") as string;

    await addProduct(title, parseInt(price), description);
    redirect("products-db");
  }
}
```

useForm Status

```
import { useFormStatus } from "react-dom";

export const Submit = () => {
  const { pending } = useFormStatus();

  return [
    <button
      type="submit"
      className="block w-full p-2 text-white bg-blue-500 rounded disabled:bg-gray-500"
      disabled={pending}
    >
      Submit
    </button>
  ];
};
```

useActionState

```
import { Submit } from "@components/submit";
import { addProduct } from "@prisma-db";
import { redirect } from "next/navigation";
import { useActionState } from "react";
```

```
type Errors = {
  title?: string;
  price?: string;
  description?: string;
};
```

```
type FormState = {
  errors: Errors;
};
```

```
export default function AddProductPage() {
  const initialState: FormState = {
    errors: {},
  };

  useActionState()
```

```
useActionState(createProduct, initialState);
```

```
async function createProduct(formData: FormData) {
  "use server";
```

```
const descriptionI = formData.get("description") as string;

const errors: Errors = {};

if (!title) {
  errors.title = "Title is required";
}

if (!price) {
  errors.price = "Price is required";
}

if (!description) {
  errors.description = "Description is required";
}

await addProduct(title, parseInt(price), description);
redirect("/products-db");
```

```
if (!description) {
  errors.description = "Description is required";
}

if (Object.keys(errors).length > 0) {
  return { errors };
}
```

```
const [state, formAction, isPending] = useActionState(
  createProduct,
  initialState
);
```

```

    state.errors.title && <p>{state.errors.title}</p>
  }
  <label className="text-white">
    Price
    <input
      type="number"
      className="block w-full p-2 text-black border rounded"
      name="price"
    />
  </label>

```

```

<form action={formAction} className="p-4 space-y-4 max-w-96">

```

useOptimistic hook

useOptimistic Hook

useOptimistic is a React Hook that provides a way to optimistically update the UI while an asynchronous action is underway

This technique helps make our apps feel more responsive, especially when working with forms

Instead of making users wait for server responses, we can show them the expected result right away.

```

import { useOptimistic } from "react";
import { removeProduct } from "@actions

```

```

export default async function ProductsDBPage() {
  const products: Product[] = await getProducts();
  const [optimisticProducts, ] = useOptimistic(products, (currentProducts, productId) => {
    return currentProducts.filter((product) => product.id !== productId);
  });
  return /

```

Use optimistic product in loop

```

return (
  <ul className="space-y-4 p-4">
    {optimisticProducts.map((product) => (
      <li
        key={product.id}
        className="p-4 bg-white shadow-md rou

```

Show imitiate delete change even before deletion complete on the background

```

const removeProductById = async (productId: number) => {
  setOptimisticProducts(productId);
  await removeProduct(productId);
};

```

Use deleted function on the form action

```
<p className="text-lg font-medium">${product.p  
<form action={removeProductById.bind([null, product.id])}>  
property) id: number ×  id  
description  
className="px-4 py-2 mt-4  text-white  bg-red-500 rounded-  
>  
Delete  
</button>  
</form>
```